Apache Hadoop End-to-End Field Manual

Complete Practical Lab Guide: Clean WSL Reset → Ubuntu → Java → Hadoop → Config → MapReduce

Target OS: Windows 10/11 with WSL2 (Ubuntu 22.04 / 24.04 LTS)	**Apache Hadoop Core:** Hadoop 2.10.x / 3.3.6 (Pseudo-Distributed)
Java Runtime: OpenJDK 11 LTS (/usr/lib/jvm/java-11-openjdk-amd64)	**Cluster User:** Dedicated hduser in hadoop group
Configuration Base: /usr/local/hadoop & /usr/local/hadoop_tmp	**Lab Verification:** 5 Hadoop Daemons via jps + WordCount MapReduce

1. Windows Subsystem for Linux (WSL) Clean Reset & Fresh Reinstall

To guarantee an immaculate, conflict-free Hadoop environment, open **PowerShell as Administrator** on Windows and execute these commands to check, unregister/wipe existing distributions, and fresh install WSL with Ubuntu:

Step 1.1: Check Existing WSL Status & List Registered Distros

```
# 1. Check WSL operational status and default version
wsl --status

# 2. List all existing distributions, their state and WSL version
wsl --list --verbose
```

Step 1.2: Shut Down and Delete / Unregister Old Ubuntu Distro

Completely terminate background instances and unregister old distributions to clear corrupted disks, packages, or stale port bindings:

```
# 1. Shut down all running WSL processes
wsl --shutdown

# 2. Delete and unregister existing Ubuntu distribution (Complete Wipe)
wsl --unregister Ubuntu

# (If you have specific versions installed, unregister them as well):
# wsl --unregister Ubuntu-22.04
# wsl --unregister Ubuntu-20.04
```

Why Unregister First?

Running wsl --unregister Ubuntu permanently deletes the virtual root disk (ext4.vhdx). This guarantees that prior broken Java installations, orphaned Hadoop lockfiles, or duplicate SSH keys cannot interfere with your fresh setup.

Step 1.3: Fresh Reinstallation of WSL & Ubuntu

```
# 1. Update the Windows Subsystem for Linux kernel component
wsl --update

# 2. Set default WSL architecture to Version 2
wsl --set-default-version 2

# 3. Freshly install Ubuntu from official Microsoft cloud image
wsl --install -d Ubuntu --web-download
```

Note: If Windows prompts for a system restart to enable Hyper-V / Virtual Machine Platform, reboot your PC before launching Ubuntu.

2. Linux (Ubuntu) Initialization & User Setup

Launch the freshly installed Ubuntu shell, configure your initial UNIX credentials, and establish the dedicated hduser and hadoop user group as required by standard Hadoop lab specifications.

Step 2.1: Launch Ubuntu & Create Initial Admin User

```
# Launch newly installed Ubuntu shell from PowerShell or Terminal
wsl -d Ubuntu
```

Enter your primary UNIX username (e.g., friday or ubuntu) and password when prompted.

Step 2.2: Create Dedicated Hadoop Group & User (hduser)

As specified in teacher lab notes, isolate Hadoop services under a dedicated service account:

```
# 1. Create the hadoop user group
sudo addgroup hadoop

# 2. Add dedicated 'hduser' belonging to the hadoop group
sudo adduser --ingroup hadoop hduser

# 3. Grant sudo privileges to hduser
sudo adduser hduser sudo
```

3. System Update (apt update) & Prerequisites

Update the APT package manager repositories, apply system security patches, and install required networking/archive utilities:

Step 3.1: Update & Upgrade APT Repositories

```
sudo apt update -y && sudo apt upgrade -y
```

Step 3.2: Install OpenSSH Server, rsync, aria2 & Networking Tools

```
sudo apt install -y openssh-server openssh-client rsync curl wget \
tar git aria2 procps net-tools ca-certificates nano
```

4. Java Development Kit (OpenJDK 11) Installation

Apache Hadoop executes on the Java Virtual Machine. Install OpenJDK 11 LTS (or default-jdk) and verify the compiler and runtime:

Step 4.1: Install OpenJDK 11 JDK

```
# Install OpenJDK 11 (standard LTS for Hadoop 2.10.x & 3.x)
sudo apt install -y openjdk-11-jdk
```

Step 4.2: Verify Java Runtime & Detect Absolute JAVA_HOME

```
# Check Java runtime and compiler versions
java -version
javac -version

# Confirm exact filesystem path to JAVA_HOME (typically /usr/lib/jvm/java-11-openjdk-amd64)
readlink -f $(which java) | sed 's:/bin/java::'
```

5. Passwordless SSH Configuration for hduser

Hadoop scripts (such as start-dfs.sh and start-yarn.sh) manage master/worker daemons over SSH. Configure key-based passwordless access to localhost:

Step 5.1: Switch to hduser and Generate SSH Keypair

```
# 1. Switch to hduser session and go to home directory
su - hduser
cd ~

# 2. Generate RSA keypair without passphrase (-P "")
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa -q

# 3. Append public key to authorized_keys and apply strict permissions
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Step 5.2: Start SSH Service & Test Connection to localhost

```
# 1. Ensure SSH server daemon is running
sudo service ssh start

# 2. Test passwordless SSH login to localhost (should connect without prompting for password)
ssh -o StrictHostKeyChecking=no localhost "echo SSH_SUCCESS"

# If successful, exit back to shell:
# exit
```

6. Apache Hadoop Download & Installation (/usr/local/hadoop)

Download Apache Hadoop (either Hadoop 2.10.2 / 2.7.1 as in teacher's notes, or stable 3.3.6) using 16 concurrent connections, extract to /usr/local/hadoop, and set ownership:

Step 6.1: High-Speed 16-Thread Download via aria2c / curl

```
# Option A: Download Hadoop 3.3.6 (Modern Stable) with 16 threads
aria2c -x 16 -s 16 -k 1M -d /tmp -o hadoop.tar.gz \
https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

# Option B: Download Hadoop 2.10.2 (Teacher Lab Package from Archive):
# aria2c -x 16 -s 16 -k 1M -d /tmp -o hadoop.tar.gz \
# https://archive.apache.org/dist/hadoop/common/hadoop-2.10.2/hadoop-2.10.2.tar.gz
```

Step 6.2: Extract Tarball to /usr/local/hadoop & Assign Permissions

```
# 1. Create destination directory and extract archive
sudo mkdir -p /usr/local/hadoop
sudo tar -xzf /tmp/hadoop.tar.gz -C /usr/local/hadoop --strip-components=1

# 2. Assign ownership of /usr/local/hadoop to hduser:hadoop
sudo chown -R hduser:hadoop /usr/local/hadoop
sudo chmod -R 755 /usr/local/hadoop
```

Step 6.3: Create Dedicated Storage Directories (/usr/local/hadoop_tmp)

Create the HDFS data directories specified in the teacher's configuration file:

```
# Create NameNode, DataNode and temporary directories
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/namenode
sudo mkdir -p /usr/local/hadoop_tmp/hdfs/datanode
sudo chown -R hduser:hadoop /usr/local/hadoop_tmp
sudo chmod -R 775 /usr/local/hadoop_tmp
```

7. Environment Variables (~/.bashrc & hadoop-env.sh)

Configure shell paths and environment bindings in ~/.bashrc and hadoop-env.sh exactly matching the teacher's configuration specification:

Step 7.1: Append Hadoop Paths to ~/.bashrc

```
# Append variables to hduser's ~/.bashrc
cat << 'EOF' >> ~/.bashrc

# >>> Apache Hadoop Lab Environment >>>
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin
export PATH=$PATH:$HADOOP_HOME/sbin
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib"
# <<< Apache Hadoop Lab Environment <<<
EOF

# Apply changes immediately
source ~/.bashrc
```

Step 7.2: Set JAVA_HOME in hadoop-env.sh

```
# Explicitly declare JAVA_HOME in hadoop-env.sh
echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64' >> /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

8. Hadoop XML Configuration Files (Teacher Specification)

Place the following configuration XML blocks between <configuration> and </configuration> tags:

Step 8.1: Configure core-site.xml

File path: /usr/local/hadoop/etc/hadoop/core-site.xml

```
cat << 'EOF' > /usr/local/hadoop/etc/hadoop/core-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://localhost:9000</value>
</property>
<property>
<name>hadoop.tmp.dir</name>
<value>/usr/local/hadoop_tmp</value>
</property>
</configuration>
EOF
```

Step 8.2: Configure hdfs-site.xml

File path: /usr/local/hadoop/etc/hadoop/hdfs-site.xml

```
cat << 'EOF' > /usr/local/hadoop/etc/hadoop/hdfs-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.namenode.name.dir</name>
<value>file:/usr/local/hadoop_tmp/hdfs/namenode</value>
</property>
<property>
<name>dfs.datanode.data.dir</name>
<value>file:/usr/local/hadoop_tmp/hdfs/datanode</value>
</property>
</configuration>
EOF
```

Step 8.3: Configure yarn-site.xml

File path: /usr/local/hadoop/etc/hadoop/yarn-site.xml

```
cat << 'EOF' > /usr/local/hadoop/etc/hadoop/yarn-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
<property>
<name>yarn.nodemanager.aux-services.mapreduce.shuffle.class</name>
<value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
<!-- WSL / Container Virtual Memory Protection -->
<property>
<name>yarn.nodemanager.pmem-check-enabled</name>
<value>false</value>
</property>
<property>
<name>yarn.nodemanager.vmem-check-enabled</name>
<value>false</value>
</property>
</configuration>
EOF
```

Step 8.4: Configure mapred-site.xml

File path: /usr/local/hadoop/etc/hadoop/mapred-site.xml (copy template if on Hadoop 2.x):

```
# If mapred-site.xml.template exists (Hadoop 2.x), copy template first:
[ -f /usr/local/hadoop/etc/hadoop/mapred-site.xml.template ] && \
cp /usr/local/hadoop/etc/hadoop/mapred-site.xml.template /usr/local/hadoop/etc/hadoop/mapred-site.xml

cat << 'EOF' > /usr/local/hadoop/etc/hadoop/mapred-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
</configuration>
EOF
```

Step 8.5: Format HDFS NameNode Storage

Initialize the distributed filesystem directory structure (execute only once before the very first cluster startup):

```
hdfs namenode -format
```

Format Verification

Look for the output line: common.Storage: Storage directory /usr/local/hadoop_tmp/hdfs/namenode has been successfully formatted. and Exiting with status 0.

9. Starting Hadoop Services & JPS Daemon Verification

Start HDFS and YARN daemons using standard scripts or the auto-stopping supervisor, then verify all 5 cluster daemons using jps:

Step 9.1: Start HDFS and YARN Services

```
# Start HDFS daemons (NameNode, DataNode, SecondaryNameNode)
start-dfs.sh

# Start YARN daemons (ResourceManager, NodeManager)
start-yarn.sh

# (Alternative: You can also run 'start-all.sh' or use 'ran-hadoop')
```

Step 9.2: Verify Active Daemons with 'jps'

```
jps

# Expected Output (All 5 Hadoop Daemons + Jps):
# [PID] NameNode
# [PID] DataNode
# [PID] SecondaryNameNode
# [PID] ResourceManager
# [PID] NodeManager
# [PID] Jps
```

Hadoop Daemon	Process Role	Web Dashboard URL
NameNode	HDFS Master: Block metadata & namespace	http://localhost:9870 (or :50070)
DataNode	HDFS Worker: Physical block storage	http://localhost:9864 (or :50075)
ResourceManager	YARN Master: Scheduling & cluster resources	http://localhost:8088
NodeManager	YARN Worker: Container lifecycle monitor	http://localhost:8042

10. Executing MapReduce WordCount Step-by-Step

Demonstrate the complete distributed MapReduce computation pipeline: create local input data, ingest into HDFS, execute the MapReduce JAR, inspect output, and download results to the local filesystem:

Step 10.1: Create Local Sample Data File

```
cat << 'EOF' > sample_input.txt
Hadoop is an open source framework for distributed storage and processing.
MapReduce is a programming model for processing big data with a parallel algorithm.
HDFS provides high-throughput access to application data.
YARN manages resources and schedules computational jobs on the cluster.
Hadoop MapReduce and HDFS make big data scalable, fault-tolerant, and reliable.
EOF
```

Step 10.2: Prepare HDFS Directories & Ingest Input Data

```
# 1. Create target input directory in HDFS
hdfs dfs -mkdir -p /my_data/input

# 2. Copy local sample file into HDFS
hdfs dfs -put -f sample_input.txt /my_data/input/

# 3. Verify directory contents and file content in HDFS
hdfs dfs -ls /my_data/input
hdfs dfs -cat /my_data/input/sample_input.txt | head -n 5
```

Step 10.3: Execute MapReduce WordCount Job

Locate and execute the Hadoop MapReduce examples JAR:

```
# 1. Ensure previous output directory is removed (MapReduce requires non-existent output dir)
hdfs dfs -rm -r -f /my_data/output

# 2. Run MapReduce WordCount Job across cluster nodes
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar \
wordcount /my_data/input /my_data/output
```

Step 10.4: Inspect Computed Output in HDFS

```
# 1. Check generated files (_SUCCESS and part-r-00000)
hdfs dfs -ls /my_data/output

# 2. Display computed word counts
hdfs dfs -cat /my_data/output/part-r-00000 | head -n 25
```

Step 10.5: Export Output Results to Local Disk / Desktop

```
# Download the output file from HDFS to local directory
hdfs dfs -get /my_data/output/part-r-00000 ./wordcount_results.txt

# Display local results file
cat wordcount_results.txt
```

Step 10.6: Safely Stop Hadoop Services

```
# Stop YARN compute daemons
stop-yarn.sh

# Stop HDFS storage daemons
stop-dfs.sh
```

11. Python Streaming MapReduce Reference

Hadoop Streaming allows any Python script to act as a MapReduce task through standard Unix STDIN and STDOUT:

The Mapper (mapper.py)

```
#!/usr/bin/env python3
import sys
for line in sys.stdin:
for word in line.strip().split():
print(f"{word}\t1")
```

The Reducer (reducer.py)

```
#!/usr/bin/env python3
import sys
current_word, current_count = None, 0
for line in sys.stdin:
word, count = line.strip().split('\t', 1)
count = int(count)
if current_word == word:
current_count += count
else:
if current_word: print(f"{current_word}\t{current_count}")
current_word, current_count = word, count
if current_word: print(f"{current_word}\t{current_count}")
```

Running Python Streaming on Hadoop:

```
chmod +x mapper.py reducer.py
hdfs dfs -rm -r -f /my_data/py_output
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-input /my_data/input \
-output /my_data/py_output \
-mapper ./mapper.py \
-reducer ./reducer.py
```

12. Master Command Cheat Sheet & Troubleshooting Matrix

Sequential cheat sheet matching every action from clean WSL setup to MapReduce job verification:

Phase / Step	Exact Terminal Command
1. Check WSL Status	wsl --status and wsl -l -v
2. Wipe / Unregister WSL	wsl --shutdown && wsl --unregister Ubuntu
3. Fresh WSL & Ubuntu Install	wsl --install -d Ubuntu --web-download
4. Launch Ubuntu Shell	wsl -d Ubuntu
5. Update APT Packages	sudo apt update -y && sudo apt upgrade -y
6. Install Dependencies	sudo apt install -y openssh-server rsync curl aria2
7. Install OpenJDK 11	sudo apt install -y openjdk-11-jdk
8. Create Hadoop User	sudo addgroup hadoop && sudo adduser --ingroup hadoop hduser
9. SSH Keypair Generation	ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
10. Authorize Public Key	cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
11. Download Hadoop (16 Threads)	aria2c -x 16 -s 16 -o hadoop.tar.gz <Archive_URL>
12. Extract to /usr/local/hadoop	sudo tar -xzf hadoop.tar.gz -C /usr/local/hadoop --strip-components=1
13. Create Cluster Temp Dirs	sudo mkdir -p /usr/local/hadoop_tmp/hdfs/{namenode,datanode}
14. Format NameNode	hdfs namenode -format
15. Start Cluster Services	start-dfs.sh && start-yarn.sh (or ran-hadoop)
16. Verify Daemons	jps (NameNode, DataNode, SecNameNode, RM, NM)
17. Ingest Input File	hdfs dfs -mkdir -p /input && hdfs dfs -put sample.txt /input/
18. Run WordCount MapReduce	hadoop jar hadoop-mapreduce-examples-*.jar wordcount /input /output
19. Inspect WordCount Results	hdfs dfs -cat /output/part-r-00000 | head -n 25
20. Stop Cluster Services	stop-yarn.sh && stop-dfs.sh

NameNode in SafeMode	HDFS is in read-only audit mode upon boot	hdfs dfsadmin -safemode leave
Port 9000 / 9870 Locked	Previous orphaned daemon holding port open	kill -9 $(lsof -t -i:9000)
Output Dir Already Exists	Hadoop refuses overwrite for data integrity	hdfs dfs -rm -r -f <path>
YARN Container Memory Fail	WSL virtual memory calculation mismatch	Add pmem-check-enabled=false to yarn-site.xml